

DD_SUB-WF-FRAMEWORK-01-WORKERS - Shared Workflow Workers, Messages, User Tasks, Drools

Developer implementation guide: DD_SUB-WF-FRAMEWORK-01-WORKERS-DEV_IMPL_GUIDE-v1.0.md.

MVP simplification baseline: worker names in this DD are logical task topics. They are not deployment names. First release should run a consolidated subscription worker application with per-topic metrics, logs, retries, and dead-letter visibility. Split topics into separate deployments only for throughput, dependency isolation, security, or team ownership.

1. Purpose

This DD defines the shared worker catalog used by subscription workflow BPMNs.

The parent DD, SUB-WF-FRAMEWORK-01, owns operation orchestration tables and operation APIs. This companion DD owns the reusable workflow building blocks:

- Camunda external task worker topics.
- Worker input and output process variables.
- Shared Camunda messages.
- Shared Camunda user tasks.
- Drools rule package naming.
- FOUNDATION_AUTH groups used by workflow tasks and trigger endpoints.
- Worker deployment and runtime rules.

Use this DD when implementing:

- A worker pod that fetches Camunda external tasks.
- A BPMN service task that references a worker topic.
- A BPMN message catch event.
- A BPMN user task.
- A Drools package called from subscription workflows.
- Worker runtime configuration, retry behavior, and scaling.

2. Ownership

This DD owns:

- The list of approved shared worker topics.
- The contract for every worker topic: purpose, when to use it, inputs, outputs, and failure behavior.
- Shared message names used by subscription BPMNs.
- Shared user task definitions and required form fields.
- Shared Drools KJAR/package naming.
- Worker implementation rules.
- Worker deployment and infrastructure recommendations.

This DD does not own:

- subscription_operation schema, idempotency, operation APIs, or outbox schema. See SUB-WF-FRAMEWORK-01.
- Subscription state and internal mutation API implementation. See SUB-LM-01.
- Billing implementation. See BIL DDs.
- Fulfillment implementation. See FUL DDs.
- Notification implementation. See NOT-01.
- Equipment lifecycle implementation. See OSR-INSTANCE-01.

- Camunda platform installation. See FOUNDATION_CAMUNDA.
- Drools platform installation. See FOUNDATION_DROOLS.
- Auth platform installation. See FOUNDATION_AUTH.

3. Mental Model

Subscription BPMNs are composed from small reusable workers.

```
flowchart LR
    BPMN[Camunda BPMN] -->|external task topic| W[Worker Pod]
    W -->|service token| API[Owning Module API]
    W -->|complete/failure| BPMN
    API --> DB[(Module DB)]
    API --> OUTBOX[(kafka_outbox)]
```

Each worker must do one job only.

Important operational distinction:

```
Logical worker topic != dedicated pod
```

This DD defines logical worker topics because BPMNs need clear, stable contracts. It does not require one pod, one Laravel command, or one deployment per topic.

One worker application may subscribe to many related topics:

```
subscription-worker app:
- sub-lm-put-pending-status
- sub-lm-commit-state
- sub-lm-revert-to-prior
- sub-lm-put-master-fields
- subscription-operation-finalize

integration-worker app:
- bil01-charge-pre-authorize
- bil01-confirm-charge
- bil01-revert-pending-charge
- ful-call-activate
- ful-call-terminate
```

The rule is that the topic contract stays single-purpose. The runtime process may consolidate several topics for easier operations and monitoring.

Good worker design:

```
sub-lm-commit-state
- calls SUB-LM commit-state API
- writes returned variables to Camunda
- fails only for commit-state errors
```

Bad worker design:

```
subscription-operation-handler
- checks operator
- decides billing
- calls fulfillment
- sends notification
- emits Kafka
```

Operator-specific behavior belongs in BPMN, Drools, configuration, and auth groups. It must not be hard-coded inside workers.

4. Worker Runtime Contract

All workers use Camunda external tasks.

Worker loop:

1. Fetch and lock tasks by topic.
2. Read process variables.
3. Validate required variables.
4. Call the owning module API or local service.
5. Complete the task with output variables.

6. On retryable failure, report failure to Camunda with remaining retries.

7. On non-retryable business failure, throw BPMN error or complete with a decision variable, depending on BPMN design.

Camunda calls:

```
POST /engine-rest/external-task/fetchAndLock
POST /engine-rest/external-task/{taskId}/complete
POST /engine-rest/external-task/{taskId}/failure
POST /engine-rest/external-task/{taskId}/bpmnError
```

Worker implementation can be Java/Spring, Laravel/PHP, or another approved stack. The topic contract must stay the same.

5. Worker Implementation Rules

Rule	Description
R-WORKER-1	A logical worker topic must represent one business step. A single worker application/process may handle multiple topics if each topic keeps its own handler, metrics, retry policy, and contract.
R-WORKER-2	A worker must not branch on <code>operator_code</code> for business behavior. Use BPMN, Drools, or config instead.
R-WORKER-3	A worker must validate required input variables before calling downstream APIs.
R-WORKER-4	A worker must be idempotent because Camunda may retry the same external task.
R-WORKER-5	Every downstream API call must include an idempotency key derived from <code>operationId</code> and worker topic.
R-WORKER-6	A worker must set only documented output variables.
R-WORKER-7	A worker must not store large payloads in Camunda variables. Store large documents in DB/object storage and pass IDs.
R-WORKER-8	A worker must not write directly to another module database. Call the owning module API.
R-WORKER-9	A worker must not publish directly to Kafka unless the worker is specifically <code>kafka-emit</code> and it writes to <code>kafka_outbox</code> .
R-WORKER-10	A worker must not complete a task after the downstream call failed unless failure is explicitly reliability-isolated, such as <code>notification-send</code> .
R-WORKER-11	Worker logs must include <code>operationId</code> , <code>subscriptionId</code> , <code>processInstanceId</code> , <code>topicName</code> , and <code>externalTaskId</code> .
R-WORKER-12	Worker failures must avoid secrets, stack traces, and full external payloads in user-visible fields.

6. Shared Variables

These variables should be available in every subscription workflow process instance.

Variable	Purpose	Example
<code>operationId</code>	Links the BPMN to <code>subscription_operation.operation_id</code> .	<code>subop_01HZ_KAMAU_TERM</code>
<code>operatorCode</code>	Operator scope.	WIK
<code>subscriptionId</code>	Subscription being changed.	<code>sub_01HZ_KAMAU_FIBER</code>
<code>operationKind</code>	Operation verb.	TERMINATE
<code>initiatingActorUserId</code>	User or service account that started the operation.	<code>bo-admin-007</code>
<code>initiatingActorRole</code>	Auth role used to start the operation.	CALL_CENTER
<code>initiatingWorkflowKind</code>	Upstream trigger kind.	BO_UI
<code>initiatingWorkflowRef</code>	Optional external reference.	<code>case_88291</code>
<code>correlationId</code>	End-to-end trace/correlation ID.	<code>corr_01HZ_KAMAU_TERM</code>

Every worker may read these common variables. A worker should fail fast if a variable it requires is missing.

7. Service-Task Worker Catalog

Version 1.0 defines 24 service-task workers.

7.1 SUB-LM State Primitive Workers

These workers call SUB-LM-01 internal APIs. Use them when a BPMN needs to mutate or compute subscription state.

sub-lm-put-pending-status

What it is for:

Moves a subscription into a transient PENDING_* status and records the active transition type.

When to use it:

- At the start of activation, pause, resume, upgrade, downgrade, relocation, migration, or termination flows.
- Before external billing/fulfillment steps when the subscription must show that a lifecycle change is in progress.

Inputs:

Variable	Meaning	Example
subscriptionId	Subscription to update.	sub_01HZ_KAMAU_FIBER
toStatus	Pending status to set.	PENDING_TERMINATION
currentTransitionType	Operation type in progress.	TERMINATE
currentTransitionReasonCode	Reason for the transition.	CUSTOMER_MOVED_NO_COVERAGE

Outputs:

Variable	Meaning	Example
priorStatusSnapshot	Prior status data used for revert.	{"statusCode": "ACTIVE", "activeRestrictions": []}
lmEventId	SUB-LM low-level event/outbox ID.	evt_01HZ_PENDING_TERM

Failure behavior:

- Retry on transient SUB-LM API/DB errors.
- BPMN error on invalid transition.

sub-lm-commit-state

What it is for:

Commits the final subscription status after the saga has succeeded.

When to use it:

- End of activation to move PENDING_ACTIVATION to ACTIVE.
- End of pause or suspend-NP to move pending status to SUSPENDED.
- End of termination to move PENDING_TERMINATION to TERMINATED.

Inputs:

Variable	Meaning	Example
subscriptionId	Subscription to update.	sub_01HZ_KAMAU_FIBER
toStatus	Final status.	TERMINATED
expectedFromStatus	Guard status expected before commit.	PENDING_TERMINATION

Outputs:

Variable	Meaning	Example
lmEventId	SUB-LM event/outbox ID.	evt_01HZ_TERM_COMMIT
committedAt	Commit timestamp.	2026-07-20T14:30:18Z

Failure behavior:

- Retry on transient SUB-LM errors.
- BPMN error if current status is not expectedFromStatus.

sub-lm-revert-to-prior

What it is for:

Reverts a subscription from a PENDING_* state back to its prior status after a workflow failure or cancellation.

When to use it:

- In BPMN compensation paths.
- After billing or fulfillment failure when the subscription must return to the status it had before the operation.

Inputs:

Variable	Meaning	Example
subscriptionId	Subscription to revert.	sub_01HZ_KAMAU_FIBER
priorStatus	Status to restore.	ACTIVE
failureReasonCode	Machine-readable failure reason.	FULFILLMENT_FAILED

Outputs:

Variable	Meaning	Example
lmEventId	SUB-LM event/outbox ID.	evt_01HZ_REVERT
revertedAt	Revert timestamp.	2026-07-20T14:31:02Z

Failure behavior:

- Retry on transient errors.
- If revert fails repeatedly, route to manual-recovery-decision.

sub-lm-put-master-fields

What it is for:

Updates selected subscription master fields as part of a workflow.

When to use it:

- Termination sets terminated_at.
- Relocation sets previous_homepass_id and new_homepass_id.
- Upgrade/downgrade sets previous_package_ref and new_package_ref.

Inputs:

Variable	Meaning	Example
subscriptionId	Subscription to update.	sub_01HZ_KAMAU_FIBER
fieldsJson	Explicit field changes.	{"terminatedAt": "2026-07-20T14:30:18Z"}

Outputs:

Variable	Meaning	Example
lmEventId	SUB-LM event/outbox ID.	evt_01HZ_MASTER_FIELDS

Failure behavior:

- Retry on transient errors.
- BPMN error if fieldsJson tries to update immutable fields.

sub-lm-put-active-restrictions

What it is for:

Updates the active restriction list on a subscription.

When to use it:

- RESTRICT add.
- RESTRICT remove.

- Suspend/terminate flows that clear restrictions through SUB-LM.

Inputs:

Variable	Meaning	Example
subscriptionId	Subscription to update.	sub_01HZ_NJERI_TRIPLEPLAY
restrictionsOperationKind	ADD, REMOVE, or REPLACE.	ADD
restrictionCode	Restriction code being changed.	OUTGOING_VOICE_BARRED
restrictionMetadata	Reason and fulfillment metadata.	{"reasonCode": "DUNNING_LEVEL_2"}

Outputs:

Variable	Meaning	Example
lmEventId	SUB-LM event/outbox ID.	evt_01HZ_RESTRICT
newActiveRestrictions	Full active restriction array after update.	[{"code": "OUTGOING_VOICE_BARRED"}]

Failure behavior:

- Retry on transient errors.
- BPMN error if restriction code is unknown or duplicate.

sub-lm-compute-cycle-window

What it is for:

Calculates the billing/service cycle window for a subscription and date.

When to use it:

- Before billing intent calls.
- Before proration, settlement, refund, or fee decisions.

Inputs:

Variable	Meaning	Example
subscriptionId	Subscription whose cycle is needed.	sub_01HZ_KAMAU_FIBER
referenceDate	Date inside the target cycle.	2026-07-20

Outputs:

Variable	Meaning	Example
cycleWindowJson	Cycle start/end and position.	{"cycleStart": "2026-07-01", "cycleEnd": "2026-07-31", "daysIntoCycle": 20}

Failure behavior:

- Retry on transient errors.
- BPMN error if subscription cycle fields are invalid.

sub-lm-cancel-in-flight-operation

What it is for:

Requests cancellation of an existing in-flight operation, usually so termination can interrupt it.

When to use it:

- TERMINATE arrives while another non-TERMINATE operation is in progress.
- Admin cancellation flow needs to signal a BPMN to compensate.

Inputs:

Variable	Meaning	Example
subscriptionId	Subscription with the in-flight operation.	sub_01HZ_KAMAU_FIBER
targetOperationId	Operation to cancel.	subop_01HZ_PAUSE_001
cancelReason	Machine-readable cancellation reason.	TERMINATE_INTERRUPTION

Outputs:

Variable	Meaning	Example
cancelledOperationId	Operation that received the cancel message.	subop_01HZ_PAUSE_001
cancelledAt	Timestamp when cancel was requested.	2026-07-20T14:20:00Z

Failure behavior:

- Retry if Camunda message correlation fails temporarily.
- BPMN error if target operation is already terminal.

7.2 Billing Workers

These workers call BIL modules. Use them when a subscription operation needs billing intent, pay-first charge handling, charge confirmation, or charge reversal.

Worker topic	What it is for	When to use it	Inputs	Outputs
bil01-charge-pre-autho rize	Creates a pending billable event and pay-first gate.	Before activation/upgrade or other chargeable operation that must be paid before continuation.	subscriptionId, chargeCode, customerChargeAmount, currency, chargeContext	billableEventId, chargeCreatedAt
bil01-confirm-charge	Confirms a previously pending charge.	After the operation succeeds and the charge should become final.	billableEventId	confirmedAt, chargeStatus
bil01-revert-pending-c harge	Cancels a pending charge.	Compensation path after an operation fails before confirmation.	billableEventId, cancelReason	cancelledAt
bil01-emit-intent	Asks BIL-01 to compute a structured billing decision.	Termination settlement, upgrade/downgrade proration, refund, debt write-off decisions.	subscriptionId, intentKind, intentContext, cycleWindow	intentDecisionId, intentDecision

Implementation rules:

- Always pass an idempotency key such as operationId + ':' + topicName.
- Billing money calculation remains in BIL. Workers must not compute money locally.
- Retry transient BIL errors.
- Route business declines according to the satellite BPMN.

Sample bil01-emit-intent input:

```
{
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "intentKind": "TERMINATION_INTENT",
  "intentContext": {
    "terminationTrigger": "CUSTOMER_REQUESTED",
    "terminationReasonCode": "CUSTOMER_MOVED_NO_COVERAGE",
    "coolingOffApplicable": false
  },
  "cycleWindow": {
    "cycleStart": "2026-07-01",
    "cycleEnd": "2026-07-31",
    "daysIntoCycle": 20
  }
}
```

Sample meaning:

- The workflow is asking billing to calculate termination settlement.
- Billing receives the subscription, trigger context, and cycle dates.
- Billing returns decisions such as refund, fee, write-off marker, or line items.

7.3 Fulfillment Workers

These workers call FUL modules. Use them when the workflow must execute or reverse network/service changes.

Worker topic	What it is for	When to use it	Inputs	Outputs
ful-call-activate	Calls FUL-03 service activation.	Activation BPMN after validation/payment/equipment readiness.	subscriptionId, activationCorrelationId, equipmentBindings, policy	activationFulfillmentId, activationTerminalState, activationServices
ful-call-restrict	Calls FUL-04 restriction fulfillment.	Apply/lift restriction or suspend service for non-payment.	subscriptionId, intent, restrictionCode, fulfillmentAction, reasonCategory, dunningEscalationLevel, remainingActiveRestrictions, fulfillmentCorrelationId	restrictionFulfillmentId, restrictionTerminalState, restrictionServices
ful-call-terminate	Calls FUL-05 termination fulfillment.	Termination BPMN when service must be disconnected.	subscriptionId, reasonCategory, equipmentDisposition, equipmentBindings, fulfillmentCorrelationId	terminationFulfillmentId, terminationTerminalState, terminationServices
ful-call-claim-homepass	Calls FUL-07 HomePass claim.	Activation/relocation/migration when a HomePass must be reserved for the subscription.	subscriptionId, homepassId, claimReason, fulfillmentCorrelationId	homepassClaimId, homepassClaimTerminalState, incrementedActiveTerminationPoints
ful-call-release-homepass	Calls FUL-07 HomePass release.	Termination/relocation/migration when a HomePass must be freed.	subscriptionId, homepassId, releaseReason, fulfillmentCorrelationId	homepassReleaseId, homepassReleaseTerminalState, decrementedActiveTerminationPoints

Implementation rules:

- Fulfillment correlation ID should include operationId and worker topic.
- Do not update subscription status directly from fulfillment workers. Use SUB-LM workers after fulfillment succeeds.
- If FUL-07 is not implemented yet, the HomePass workers may call a controlled stub only in environments where the architecture owner allows it.
- Fulfillment terminal failure must route to BPMN compensation.

7.4 Equipment Lifecycle Workers

These workers call OSR-INSTANCE-01.

Worker topic	What it is for	When to use it	Inputs	Outputs
osr-equipment-bind	Binds equipment instances to a subscription.	Activation flow after installer/equipment readiness is known.	subscriptionId, equipmentBindings, bindCorrelationId	boundCount, boundInstanceRefs
osr-equipment-unbind	Releases equipment from a subscription.	Activation compensation, termination, equipment return/write-off handling.	subscriptionId, equipmentBindings, unbindCorrelationId, disposition	unboundCount

Implementation rules:

- Do not invent equipment state in the worker. OSR owns equipment lifecycle state.
- Use idempotency keys so retry does not bind/unbind twice.
- If unbind fails during compensation, route to manual-recovery-decision.

7.5 Cross-Cutting Workers

These workers are shared across operation types.

notification-send

What it is for:

Sends a notification request to NOT-01.

When to use it:

- Customer operation started/completed/failed notification.

- Admin or group notification.
- Customer reply workflows.

Inputs:

Variable	Meaning	Example
recipientKind	customer or group.	customer
recipientRef	Customer ID or group code.	cust_01HZ_KAMAU
notificationEventKind	Notification business event.	SUBSCRIPTION_TERMINATED
eventContext	Template context.	{"refundAmount":1200,"currency":"KES"}

Outputs:

Variable	Meaning	Example
notificationId	NOT-01 notification ID, when available.	not_01HZ_TERM
dispatched	Whether NOT-01 accepted dispatch.	true

Failure behavior:

- Notification failure must not fail the BPMN.
- Log failure and continue unless a satellite DD explicitly requires a hard stop.

subscription-operation-history-append

What it is for:

Appends operation-specific history in a satellite-owned history table.

When to use it:

- A satellite DD requires detailed transition history beyond subscription_operation.
- Support needs a step-by-step timeline for a specific operation kind.

Inputs:

Variable	Meaning	Example
operationId	Operation being recorded.	subop_01HZ_KAMAU_TERM
fromState	Previous operation sub-state.	COOLING_OFF_CHECK
toState	New operation sub-state.	BILLING_INTENT
transitionReason	Why the transition occurred.	COOLING_OFF_NOT_APPLICABLE
actorUserId	Actor responsible, if any.	bo-admin-007
payload	Optional detail.	{"coolingOffApplicable":false}

Outputs:

Variable	Meaning	Example
historyId	Inserted history row ID.	hist_01HZ_TERM_003
eventSequence	Sequence number in operation history.	3

Failure behavior:

- Retry transient errors.
- If history is mandatory for the satellite, fail the BPMN.

drools-decide

What it is for:

Calls the subscription workflow Drools KJAR and returns decisions.

When to use it:

- Eligibility checks.
- Role checks.

- Cooling-off decisions.
- Write-off marker decisions.
- Operator-specific policy decisions.

Inputs:

Variable	Meaning	Example
kjarPackage	Rule package to execute.	rules.subscription.terminate.wik
factsJson	Facts passed to rules.	{"status": "ACTIVE", "role": "CALL_CENTER"}
requestedDecisions	Decision names needed.	["eligibility", "coolingOff"]

Outputs:

Variable	Meaning	Example
decisionsJson	Decision results.	{"eligible": true, "coolingOffDays": 7}

Failure behavior:

- Retry Drools platform errors.
- BPMN error if rules return `eligible = false` and the BPMN expects a hard stop.

kafka-emit

What it is for:

Writes a Kafka event into `kafka_outbox`.

When to use it:

- Operation-specific domain event emission such as `SubscriptionTerminated`.
- Framework lifecycle event emission if called from BPMN.

Inputs:

Variable	Meaning	Example
topic	Kafka topic.	sophix.subscription.terminated
key	Kafka key.	sub_01HZ_KAMAU_FIBER
payloadJson	Full event payload.	{"eventType": "SubscriptionTerminated", ...}

Outputs:

Variable	Meaning	Example
outboxId	Outbox row ID.	kox_01HZ_TERM_EVENT

Failure behavior:

- Retry DB/outbox insert errors.
- Do not publish directly to Kafka in this worker.

external-system-cancel

What it is for:

Notifies a downstream system that a previously requested external action should be cancelled.

When to use it:

- Work order cancellation.
- External fulfillment request cancellation.
- Other satellite-defined downstream cancellation.

Inputs:

Variable	Meaning	Example
externalSystemKind	System to notify.	WO_01
externalRef	External request/work order ID.	wo_01HZ_INSTALL_77
cancelReason	Machine-readable reason.	OPERATION_CANCELLED

Outputs:

Variable	Meaning	Example
cancelAcknowledged	Whether system accepted cancellation.	true

Failure behavior:

- Retry transient errors.
- If cancellation cannot be confirmed, route to manual recovery if the external action can create customer impact.

subscription-operation-finalize

What it is for:

Marks the parent subscription_operation row as terminal.

When to use it:

- At the end of every BPMN success path.
- At the end of every failure compensation path.
- At the end of every cancellation compensation path.

Inputs:

Variable	Meaning	Example
operationId	Operation to finalize.	subop_01HZ_KAMAU_TERM
finalState	COMPLETED, FAILED, or CANCELLED.	COMPLETED
failureReasonCode	Required when final state is FAILED.	FULFILLMENT_FAILED
failureReasonDetail	Short failure explanation.	FUL-05 returned terminal failure

Outputs:

Variable	Meaning	Example
finalizedAt	Finalization timestamp.	2026-07-20T14:30:18Z

Failure behavior:

- Retry transient DB errors.
- If operation is already terminal, complete idempotently and do not emit duplicate lifecycle events.

8. Worker Registration

Each worker deployment may declare one topic or many topics.

For operability, prefer a small number of worker applications grouped by ownership/latency profile instead of dozens of tiny deployments.

Each worker deployment must declare:

- Topic name.
- Lock duration.
- Max tasks per fetch.
- Max worker concurrency.
- Retry count.
- Retry timeout/backoff.

- Owning module.

Example neutral configuration:

```
externalTasks:
- topic: sub-lm-put-pending-status
  owningModule: SUB-LM
  lockDurationMs: 30000
  maxTasks: 10
  concurrency: 10
  retries: 3
  retryTimeoutMs: 10000

- topic: bil01-emit-intent
  owningModule: BIL-01
  lockDurationMs: 60000
  maxTasks: 5
  concurrency: 5
  retries: 3
  retryTimeoutMs: 30000

- topic: ful-call-terminate
  owningModule: FUL-05
  lockDurationMs: 90000
  maxTasks: 5
  concurrency: 5
  retries: 3
  retryTimeoutMs: 30000
```

Recommended lock durations:

Worker type	Lock duration
SUB-LM state workers	30 seconds
Billing workers	60 seconds
Fulfillment workers	90 seconds
Notification worker	30 seconds
Drools worker	30 seconds
Kafka outbox worker	30 seconds
Finalize worker	30 seconds

Choose a lock duration longer than normal p95 execution time but short enough that another worker can recover if a pod dies.

Recommended consolidation model:

Worker app	Topics it may handle	Why
subscription-state-worker	SUB-LM state topics and subscription-operation-finalize	Same module ownership, same DB/API dependency, short calls.
billing-worker	BIL-01 topics	Same downstream dependency and money-domain monitoring.
fulfillment-worker	FUL-* topics and external cancellation	Similar long-running external calls and retry profile.
cross-cutting-worker	drools-decide, kafka-emit, notification-send, history append	Shared orchestration utilities. Split later if one topic becomes high volume.
message-correlator	Payment, fulfillment, cancellation, customer reply correlation	Event-to-Camunda correlation is operationally different from service tasks.

Monitoring must still be per topic even if deployment is consolidated. Required labels:

```
worker_app
topic_name
operation_kind
operator_code
result
```

9. Concrete Worker Example

Example: bil01-emit-intent.

Camunda fetch:

```
POST /engine-rest/external-task/fetchAndLock HTTP/1.1
Content-Type: application/json
```

```
{
  "workerId": "subwf-worker-pod-02",
  "maxTasks": 1,
  "topics": [
    {
      "topicName": "bil01-emit-intent",
      "lockDuration": 60000
    }
  ]
}
```

Camunda returns:

```
[
  {
    "id": "extt_01HZ_KAMAU_TERM_BIL",
    "topicName": "bil01-emit-intent",
    "processInstanceId": "pi_01HZ_KAMAU_TERM",
    "variables": {
      "operationId": {
        "value": "subop_01HZ_KAMAU_TERM",
        "type": "String"
      },
      "subscriptionId": {
        "value": "sub_01HZ_KAMAU_FIBER",
        "type": "String"
      },
      "intentKind": {
        "value": "TERMINATION_INTENT",
        "type": "String"
      },
      "intentContext": {
        "value": "{\"terminationTrigger\":\"CUSTOMER_REQUESTED\"}",
        "type": "Json"
      },
      "cycleWindow": {
        "value": "{\"cycleStart\":\"2026-07-01\",\"cycleEnd\":\"2026-07-31\"}",
        "type": "Json"
      }
    }
  }
]
```

Worker calls BIL-01:

```
POST /api/billing-intents HTTP/1.1
Authorization: Bearer <service-jwt>
Idempotency-Key: subop_01HZ_KAMAU_TERM: bil01-emit-intent
Content-Type: application/json
```

```
{
  "subscriptionId": "sub_01HZ_KAMAU_FIBER",
  "intentKind": "TERMINATION_INTENT",
  "intentContext": {
    "terminationTrigger": "CUSTOMER_REQUESTED"
  },
  "cycleWindow": {
    "cycleStart": "2026-07-01",
    "cycleEnd": "2026-07-31"
  }
}
```

Worker completes task:

```
POST /engine-rest/external-task/extt_01HZ_KAMAU_TERM_BIL/complete HTTP/1.1
Content-Type: application/json
```

```
{
  "workerId": "subwf-worker-pod-02",
  "variables": {
    "intentDecisionId": {
      "value": "bid_01HZ_KAMAU_TERM",
      "type": "String"
    },
    "intentDecision": {
      "value": "{\"refundAmount\":1200,\"currency\":\"KES\",\"writeOffMarker\":false}",
      "type": "Json"
    }
  }
}
```

Pseudocode:

```
void handleBil01EmitIntent(ExternalTask task) {
```

```

String operationId = requiredString(task, "operationId");
String subscriptionId = requiredString(task, "subscriptionId");
String intentKind = requiredString(task, "intentKind");
JsonNode intentContext = requiredJson(task, "intentContext");
JsonNode cycleWindow = requiredJson(task, "cycleWindow");

BillingIntentResponse response = bilClient.createIntent(
    new BillingIntentRequest(subscriptionId, intentKind, intentContext, cycleWindow),
    operationId + ":bil01-emit-intent"
);

camunda.complete(task.id(), Map.of(
    "intentDecisionId", response.intentDecisionId(),
    "intentDecision", response.intentDecision()
));
}

```

10. Shared Camunda Messages

These message names are shared. BPMNs may catch them by message name and correlation key.

Message name	What it is for	Sent by	Correlation key	Payload
payment-confirmed	Resume a pay-first workflow after customer payment.	BIL-01 payment event correlator.	billableEventId	billableEventId, paidAt, walletId, paidAmount
payment-timeout	Move pay-first workflow to timeout path.	Camunda boundary timer.	Process instance.	None
fulfillment-completed	Resume workflow after fulfillment success.	FUL dispatcher/correlator.	fulfillmentId	fulfillmentId, fulfillmentKind, terminalState, servicesSummary
fulfillment-failed	Resume workflow after fulfillment terminal failure.	FUL dispatcher/correlator.	fulfillmentId	fulfillmentId, fulfillmentKind, failureReason, failureDetail
external-cancel-requested	Tell an in-flight BPMN to compensate and cancel.	Framework cancel API or TERMINATE interrupt flow.	operationId	cancelReason, cancelActorUserId
bpmn-cancel-requested	Backward-compatible cancel message name for admin cancel.	Framework cancel API.	operationId	cancelReason, cancelActorUserId
customer-reply-received	Resume workflow waiting for customer SMS/portal reply.	NOT-01 inbound handler.	subscriptionId or operation-specific reply token.	replyKind, subscriptionId, replyPayload

Implementation rules:

- Prefer external-cancel-requested for new BPMNs.
- Message correlators must be idempotent.
- If a message arrives before the BPMN is waiting, store it and retry correlation.
- Never drop unmatched messages silently.

Sample fulfillment-completed message:

```

{
  "messageName": "fulfillment-completed",
  "correlationKey": "ful_01HZ_TERM_001",
  "payload": {
    "fulfillmentId": "ful_01HZ_TERM_001",
    "fulfillmentKind": "TERMINATE",
    "terminalState": "COMPLETED",
    "servicesSummary": {
      "internet": "DISCONNECTED",
      "voice": "DISCONNECTED"
    }
  }
}

```

Sample meaning:

- FUL-05 completed termination fulfillment.
- The BPMN waiting on ful_01HZ_TERM_001 can continue.

- The BPMN can now commit SUB-LM final state and emit domain events.

11. Shared User Tasks

User tasks are Camunda human tasks completed through module/UI APIs. Users should not call Camunda directly from arbitrary clients.

User task ID	What it is for	Candidate group	Required form fields	Used by
cooling-off-override-decision	Supervisor decides whether to override cooling-off policy.	COOLING_OFF_SUPERVISORS	overrideApproved, overrideReason, notes	Termination flows.
churn-save-attempt	Agent records retention attempt result.	CHURN_SAVE_AGENT	saveAttempted, customerAccepted, offerCode, notes	Customer-initiated termination.
manual-recovery-decision	Ops decides how to recover a stuck operation.	SUBSCRIPTION_OPS_RECOVERY	recoveryAction, notes	Any flow stuck in compensation/recovery.
finance-write-off-approval	Finance approves high-value debt write-off.	FINANCE_WRITE_OFF_APPROVAL	writeOffApproved, writeOffAmount, notes	Dunning termination.
customer-self-service-confirm	Customer confirms settlement or final action.	CUSTOMER_SELF_SERVICE	customerConfirmedSettlement	Self-service flows requiring confirmation.

Field descriptions:

Field	Purpose	Example
overrideApproved	Whether supervisor approved the override.	true
overrideReason	Why override was approved or rejected.	CUSTOMER_ESCALATION
saveAttempted	Whether a churn-save attempt happened.	true
customerAccepted	Whether customer accepted the save offer.	false
offerCode	Offer presented to customer.	RETENTION_20_PERCENT_3M
recoveryAction	Manual recovery decision: RETRY, FORCE_COMPLETE, or FORCE_REVERT.	RETRY
writeOffApproved	Whether finance approved write-off.	true
writeOffAmount	Amount approved for write-off.	15000.00
customerConfirmedSettlement	Whether customer accepted self-service settlement.	true
notes	Human-readable explanation.	Customer confirmed via recorded call.

Implementation rules:

- Candidate groups are logical group codes. FOUNDATION_AUTH maps them to operator-specific groups.
- UI must validate required fields before completing the task.
- Completion API must verify that the user belongs to the resolved candidate group.
- Form payload must be stored in audit/history, not only in Camunda variables.

12. Shared Drools KJAR

One KJAR is used for subscription workflow rules:

subscription-workflow-rules

Package naming:

Package	What it is for	Operator-scoped?
rules.subscription.common	Cross-operation shared validation.	No
rules.subscription.<kind>.<operator>	Operation/operator policy.	Yes

Examples:

Package	Meaning
rules.subscription.activate.wik	WIK activation rules.
rules.subscription.terminate.wik	WIK termination rules.
rules.subscription.suspend_np.wik	WIK non-payment suspension rules.

Package	Meaning
rules.subscription.terminate.yassn	YASSN termination rules.

Use Drools for:

- Eligibility rules.
- Actor role validation.
- Cooling-off decisions.
- Write-off thresholds.
- Operator-specific policy values.
- Routing decisions that change often by operator.

Do not use Drools for:

- Simple null checks.
- Database writes.
- HTTP calls.
- Money calculation owned by billing.
- Subscription state mutation.

Sample facts:

```
{
  "operatorCode": "WIK",
  "operationKind": "TERMINATE",
  "subscriptionStatus": "ACTIVE",
  "initiatingActorRole": "CALL_CENTER",
  "terminationReasonCode": "CUSTOMER_MOVED_NO_COVERAGE",
  "daysSinceActivation": 190,
  "outstandingDebtAmount": 0,
  "currency": "KES"
}
```

Sample decisions:

```
{
  "eligible": true,
  "allowedRole": true,
  "coolingOffApplicable": false,
  "requiresFinanceWriteOffApproval": false,
  "nextRoute": "BILLING_INTENT"
}
```

Sample meaning:

- WIK termination rules accepted the request.
- The caller role is allowed.
- No cooling-off approval is needed.
- Finance approval is not needed.
- BPMN can route to billing intent.

13. FOUNDATION_AUTH Groups

These logical group codes are used by subscription workflows.

Group code	What it is for	Used by
SUBSCRIPTION_ADMIN	Admin override authority.	Trigger endpoints, manual recovery.
SUBSCRIPTION_OPERATOR	Day-to-day operator authority.	Trigger endpoints.
CALL_CENTER	Customer-facing intake authority.	Customer-requested operations.
CUSTOMER_SELF_SERVICE	Customer self-service authority.	Customer portal flows and confirmation tasks.
BILLING_INTERNAL	Internal dunning/billing service authority.	BIL-04 dunning triggers.
SCHEDULER_INTERNAL	Internal scheduler authority.	Contract expiry and scheduled changes.

Group code	What it is for	Used by
FINANCE_WRITE_OFF_APPROVER	High-value debt write-off approval.	Finance write-off user task.
CHURN_SAVE_AGENT	Retention/save attempt authority.	Churn-save user task.
SUBSCRIPTION_OPS_RECOVERY	Stuck-operation recovery authority.	Manual recovery user task.
COOLING_OFF_SUPERVISORS	Cooling-off override authority.	Cooling-off override user task.

Operator-specific mapping examples:

Logical group	WIK mapping	YASSN mapping
CALL_CENTER	wik-call-center-tier1	yassn-call-center-tier1
COOLING_OFF_SUPERVISORS	wik-cooling-off-supervisors	yassn-call-center-supervisors
SUBSCRIPTION_OPS_RECOVERY	wik-subscription-ops-recovery	yassn-subscription-ops-recovery

FOUNDATION_AUTH owns group membership and token claims. BPMNs and workers only consume resolved authority.

14. Infrastructure Recommendations

14.1 Deployment Unit Clarification

Workers are deployable processes/pods that talk to Camunda and module APIs. One process/pod can handle multiple logical worker topics.

They may be packaged in different ways:

Option	When to use	Notes
Workers inside owning module deployment	Budget-conscious start.	SUB-LM workers run with SUB-LM, billing workers with BIL, etc.
Dedicated subscription-worker-service	Cleaner scaling than embedding in APIs.	One worker service can subscribe to multiple topics.
One worker deployment per concern	Higher maturity.	Separate SUB-LM workers, billing workers, fulfillment workers, cross-cutting workers.
One deployment per topic	Only for exceptional high-volume or risky topics.	More operational overhead; avoid as the default.

Recommendation for this project:

Start medium-budget with workers grouped by owning module or concern. Split high-volume or high-risk topics later.

14.2 On-Prem Medium Deployment

Recommended starting point:

Worker group	Count	Size per pod	Notes
Subscription state worker pods	2	1 vCPU, 2 GB RAM	Consolidates SUB-LM state topics and operation finalize.
Billing worker pods	2	1 vCPU, 2 GB RAM	Consolidates BIL-01 topics.
Fulfillment worker pods	2	1-2 vCPU, 2-4 GB RAM	Consolidates FUL-* topics. Network calls may be slower; size by latency.
Cross-cutting worker pods	2	1 vCPU, 2 GB RAM	Consolidates Drools, notification, kafka-emit, history append.
Message correlator pods	2	1 vCPU, 2 GB RAM	Correlates Kafka/inbound events to Camunda messages.

Minimum budget packaging:

- 2 subscription app pods:
 - SUB-LM API
 - SUB-WF API
 - SUB-LM worker topics
 - cross-cutting worker topics

- 2 integration worker pods:
 - billing worker topics
 - fulfillment worker topics
 - message correlators

Do not run all workers as a single replica. If one pod dies, locked tasks wait until lock expiry. At least two replicas are needed for recovery.

14.3 OS and Partitions

Use the same OS baseline as the foundation bundle:

- Ubuntu Server 24.04 LTS, or
- Red Hat Enterprise Linux 9.

Recommended VM partitions for worker hosts:

Mount	Suggested size	Purpose
/	40 GB	OS and base packages.
/var/log	50 GB	Worker logs and task failure investigation.
/opt/sophix	20 GB	Worker binaries/config if not fully containerized.
/var/lib/containerd or /var/lib/docker	80 GB	Container images and runtime layers.
/tmp	10 GB	Temporary runtime files. Do not store payloads here.

Worker hosts must have accurate time through NTP/chrony. Timeouts, lock durations, retries, and audit timestamps depend on it.

14.4 AWS Deployment

Recommended AWS baseline:

- Run worker pods on EKS or ECS Fargate.
- Use CloudWatch Logs for worker logs.
- Use Secrets Manager or SSM Parameter Store for service credentials.
- Use IAM roles for AWS service access when possible.
- Keep workers in private subnets.
- Route worker-to-Camunda and worker-to-module traffic through internal load balancers.

Suggested AWS alarms:

Metric	Alarm condition
External task backlog per topic	Backlog older than SLA threshold.
Worker failure rate	Failure percentage above threshold for 5 minutes.
Camunda incidents	Any sustained incident count above zero.
Message correlation failures	Any sustained failure count above zero.
Worker pod restarts	More than expected restart threshold.
Downstream API latency	p95 above worker lock-duration safety margin.

15. Cross-DD Dependencies

DD/module	Relationship
SUB-WF-FRAMEWORK-01	Parent framework. Owns operation rows, APIs, idempotency, and outbox schema.
SUB-LM-01	SUB-LM workers call subscription internal mutation APIs.
BIL DDs	Billing workers call billing APIs.
FUL DDs	Fulfillment workers call fulfillment APIs.
OSR-INSTANCE-01	Equipment workers call equipment lifecycle APIs.
NOT-01	Notification worker calls notification APIs.
FOUNDATION_CAMUNDA	External task, message, timer, and user-task runtime.
FOUNDATION_DROOLS	Drools KJAR execution runtime.
FOUNDATION_AUTH	Service auth, candidate groups, and user-task completion authorization.

DD/module	Relationship
FOUNDATION_KAFKA	Events used by outbox and message correlators.
Operation satellite DDs	Compose these workers into BPMNs.

16. Build Order

Implement in this order:

1. Define shared worker client library for Camunda fetch/complete/failure.
2. Implement common variable validation and logging helper.
3. Implement SUB-LM workers.
4. Implement subscription-operation-finalize.
5. Implement kafka-emit.
6. Implement billing workers.
7. Implement fulfillment workers.
8. Implement equipment workers.
9. Implement notification worker.
10. Implement Drools worker.
11. Implement message correlators.
12. Implement user-task completion APIs.
13. Add worker metrics and alerts.
14. Run one end-to-end BPMN, normally activation or termination.

17. Test Checklist

- [] Each worker rejects missing required variables.
- [] Each worker passes documented idempotency key to downstream APIs.
- [] Each worker completes Camunda task with only documented output variables.
- [] Retryable downstream failures call Camunda failure with retries.
- [] Non-retryable business failures route correctly.
- [] Notification failure does not fail the BPMN.
- [] subscription-operation-finalize is idempotent.
- [] kafka-emit writes outbox row and does not publish directly.
- [] Message correlators handle duplicate messages safely.
- [] Message correlators persist unmatched messages for retry.
- [] User-task completion validates candidate group membership.
- [] Drools worker calls the correct package for operator and operation.
- [] Worker logs include operation/process/task correlation fields.
- [] Worker metrics expose task count, duration, success, failure, and retry count.

18. Developer Checklist

- [] Do not add operator-specific branching inside workers.
- [] Keep workers single-purpose.
- [] Keep large payloads out of Camunda variables.
- [] Use service tokens for downstream APIs.

- [] Use `operationId:topicName` idempotency keys.
- [] Keep BPMN routing decisions in BPMN/Drools, not worker code.
- [] Keep worker configuration outside code where possible.
- [] Add metrics before production deployment.
- [] Add runbook entries for Camunda incidents and stuck tasks.